In this assignment I use AI, to help me guide throughout the project step by step and help me solve problems along the way if it isn't working. First, I describe the project and started with:

"Here's a **clear step-by-step implementation plan** for enhancing your JWKS server with AES encryption, user registration, and authentication logging."

🔐 1. Project Setup (Baseline)

Before adding features:

Install dependencies

If you're using Python (Flask/FastAPI), install:

pip install flask pyjwt cryptography argon2-cffi python-dotenv uuid

If Node.js, tell me—I can convert everything.

🔑 2. AES Encryption for Private Keys

Step 2.1: Load encryption key from environment

Create `.env`:

NOT_MY_KEY=your_32_byte_secret_key_here

Load it:

```
import os
from dotenv import load_dotenv

load_dotenv()
AES_KEY = os.getenv("NOT_MY_KEY").encode()
```

⚠️ Must be 16, 24, or 32 bytes for AES.

Step 2.2: Create AES encrypt/decrypt helper

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

def encrypt_private_key(private_key: str):
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(AES_KEY), modes.CFB(iv))
    encryptor = cipher.encryptor()

    ciphertext = encryptor.update(private_key.encode()) + encryptor.finalize()
    return iv, ciphertext


def decrypt_private_key(iv, ciphertext):
    cipher = Cipher(algorithms.AES(AES_KEY), modes.CFB(iv))
    decryptor = cipher.decryptor()

    return (decryptor.update(ciphertext) + decryptor.finalize()).decode()
```

Step 2.3: Store IV separately in DB

When saving keys:

- store `encrypted_key`
- store `iv`

👤 3. Create Users Table

Run this SQL:

```
CREATE TABLE IF NOT EXISTS users(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    email TEXT UNIQUE,
    date_registered TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP
);
```

📝 4. User Registration Endpoint (POST /register)

Step 4.1: Generate UUID password

```
import uuid
from argon2 import PasswordHasher

ph = PasswordHasher()
```

Step 4.2: Register endpoint

```
from flask import request, jsonify

@app.route("/register", methods=["POST"])
def register():
  data = request.get_json()

  username = data["username"]
  email = data["email"]

  # Generate password
  raw_password = str(uuid.uuid4())

  # Hash password using Argon2
  password_hash = ph.hash(raw_password)

  # Store in DB
  conn = get_db()
  cursor = conn.cursor()

  cursor.execute("""
    INSERT INTO users (username, password_hash, email)
    VALUES (?, ?, ?)
  """, (username, password_hash, email))

  conn.commit()

  return jsonify({"password": raw_password}), 201
```

🔐 5. Authentication Logging Table

Step 5.1: Create table

```
CREATE TABLE IF NOT EXISTS auth_logs(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  request_ip TEXT NOT NULL,
  request_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  user_id INTEGER,
  FOREIGN KEY(user_id) REFERENCES users(id)
);
```

📡 6. Log Each /auth Request

Step 6.1: Helper function

```
def log_auth(request_ip, user_id):
  conn = get_db()
  cursor = conn.cursor()

  cursor.execute("""
     INSERT INTO auth_logs (request_ip, user_id)
     VALUES (?, ?)
  """, (request_ip, user_id))

  conn.commit()
```

Step 6.2: Modify `/auth` endpoint

```
@app.route("/auth", methods=["POST"])
def auth():
  data = request.get_json()
  username = data["username"]
  password = data["password"]

  conn = get_db()
  cursor = conn.cursor()

  cursor.execute("SELECT id, password_hash FROM users WHERE username=?",
(username,))
  user = cursor.fetchone()

  ip = request.remote_addr

  if not user:
     log_auth(ip, None)
```

```
        return jsonify({"error": "invalid credentials"}), 401

    user_id, password_hash = user

    try:
        ph.verify(password_hash, password)
        log_auth(ip, user_id)
        return jsonify({"status": "success"}), 200
    except:
        log_auth(ip, user_id)
        return jsonify({"error": "invalid credentials"}), 401
```

🚦 7. (Optional) Rate Limiter

Simple in-memory version:

```
from collections import defaultdict
import time

requests = defaultdict(list)
LIMIT = 5  # requests
WINDOW = 60  # seconds

def rate_limited(ip):
    now = time.time()
    requests[ip] = [t for t in requests[ip] if now - t < WINDOW]

    if len(requests[ip]) >= LIMIT:
        return True

    requests[ip].append(now)
    return False
```

Use in `/auth`:

```
if rate_limited(request.remote_addr):
    return jsonify({"error": "too many requests"}), 429
```

🧪 8. Testing Checklist

✔ Register user → get UUID password
✔ Login using password → succeeds
✔ Check DB:

- users table has hashed password
- auth_logs records every attempt
  ✔ Private keys stored encrypted
  ✔ Rate limiter blocks spam requests

A few times, I will ask AI to explain what to write in that particular file when I am stuck.

1. `app.py` (MAIN SERVER)

```
from flask import Flask, request, jsonify
import database
import auth_utils
import crypto_utils
from rate_limiter import is_rate_limited

app = Flask(__name__)

database.init_db()


# ---------------- REGISTER ----------------
@app.route("/register", methods=["POST"])
def register():
    data = request.get_json()
    username = data["username"]
    email = data["email"]

    password = auth_utils.create_user(username, email)

    return jsonify({"password": password}), 201


# ---------------- AUTH ----------------
@app.route("/auth", methods=["POST"])
def auth():
    ip = request.remote_addr
```

```
    if is_rate_limited(ip):
        return jsonify({"error": "Too many requests"}), 429

    data = request.get_json()
    username = data["username"]
    password = data["password"]

    user = auth_utils.verify_user(username, password)

    if not user:
        database.log_auth(ip, None)
        return jsonify({"error": "invalid credentials"}), 401

    database.log_auth(ip, user["id"])
    return jsonify({"status": "success"}), 200


# ---------------- JWKS ----------------
@app.route("/jwks", methods=["GET"])
def jwks():
    keys = database.get_keys()

    jwks_keys = []
    for key in keys:
        private = crypto_utils.decrypt_private_key(key["iv"], key["encrypted_key"])

        jwks_keys.append({
            "kid": key["kid"],
            "key": private,
        })

    return jsonify({"keys": jwks_keys}), 200


if __name__ == "__main__":
    app.run(debug=True)
```

Sometimes I will ask AI the error I am getting and how to fix it

“You’re running into a **PowerShell issue**, not a problem with your server.”

In **PowerShell, `curl` is NOT real curl** — it's an alias for `Invoke-WebRequest`, which uses different syntax. That's why `-X` and `-H` are failing.

✅ FIX (3 easy options)

⭐ Option 1 (BEST): Use real curl in PowerShell

Use this instead:

```
curl.exe -X POST "http://127.0.0.1:5000/register" `
-H "Content-Type: application/json" `
-d "{\"username\":\"testuser\",\"email\":\"test@email.com\"}"
```

⚠️ Important:

- Use `curl.exe` (NOT just curl)
- Use backtick instead of `^`

⭐ Option 2: Use PowerShell-native command (cleanest)

Forget curl entirely:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5000/register" `
-Method POST `
-ContentType "application/json" `
-Body '{"username":"testuser","email":"test@email.com"}'
```

👉 This is the **recommended way in Windows PowerShell**

For AUTH request:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5000/auth" `
-Method POST `
-ContentType "application/json" `
-Body '{"username":"testuser","password":"YOUR_PASSWORD"}'
```

⭐ Option 3: Use Postman (easiest overall)

If you don't want command-line issues:

- Open Postman
- POST → `http://127.0.0.1:5000/register`
- Body → JSON:

```
{
 "username": "testuser",
 "email": "test@email.com"
}
```

Click **Send**

🧠 **Why your error happened**

You did this:

curl -X POST ...

But PowerShell interprets it as:

Invoke-WebRequest -X ...

👉 And `-X` doesn't exist → error

🚀 **Recommendation (for your assignment)**

Use:
✔ `Invoke-RestMethod` (most stable)
OR
✔ Postman (best for screenshots)

"You're very close — this error is happening because **PowerShell + JSON formatting is breaking your request**, not your server.

You actually have **two separate issues**:"

❌ Problem 1: JSON format is invalid

This error:

Failed to decode JSON object: Expecting property name enclosed in double quotes

means your request body is NOT valid JSON.

In PowerShell, this usually happens when quotes are not escaped correctly.

❌ Problem 2: curl URL error

curl: (3) URL rejected: Bad hostname

This happens because PowerShell is mis-parsing your command (line breaks / quotes / curl alias confusion).

✅ BEST FIX (Use this — guaranteed to work)

⭐ Use PowerShell-native command (RECOMMENDED)

🔹 REGISTER USER

```
Invoke-RestMethod -Uri "http://127.0.0.1:5000/register" `
-Method POST `
-ContentType "application/json" `
-Body '{"username":"testuser","email":"test@email.com"}'
```

🔹 AUTH (after you get password)

```
Invoke-RestMethod -Uri "http://127.0.0.1:5000/auth" `
-Method POST `
-ContentType "application/json" `
-Body '{"username":"testuser","password":"PASTE_PASSWORD_HERE"}'
```

⚠️ IMPORTANT RULES (why yours broke)

PowerShell does NOT like:

- `curl -X`
- single-line broken JSON
- unescaped quotes
- line breaks with `^` (that's CMD, not PowerShell)

✅ If you REALLY want curl instead

You must use:

```
curl.exe -X POST "http://127.0.0.1:5000/register" `
-H "Content-Type: application/json" `
-d "{\"username\":\"testuser\",\"email\":\"test@email.com\"}"
```

🧠 Simple explanation

Tool	Problem
curl	PowerShell replaces it with Invoke-WebRequest
JSON	needs strict double quotes
`^`	only works in CMD, NOT PowerShell
`-H` / `-X`	not valid in PowerShell curl alias